

Modulo_2

Rosario Villanueva

2026-04-01

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see <http://rmarkdown.rstudio.com>.

When you click the *Knit* button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:

Primeros comandos de R

R es un lenguaje muy parecido a Matlab y permite el trabajo con matrices

```
A=38
A=40
B=60
C=B-A
C
```

```
## [1] 20
```

Uso de datasets o base de datos internos de R

Usando el comando `data` en la consola obtenemos distintos datos ya cargados en la base de datos de R, estos datos estan dados en tablas, en las cuales podemos elegir la columna que queremos ver si usamos el signo `$` despues del dato

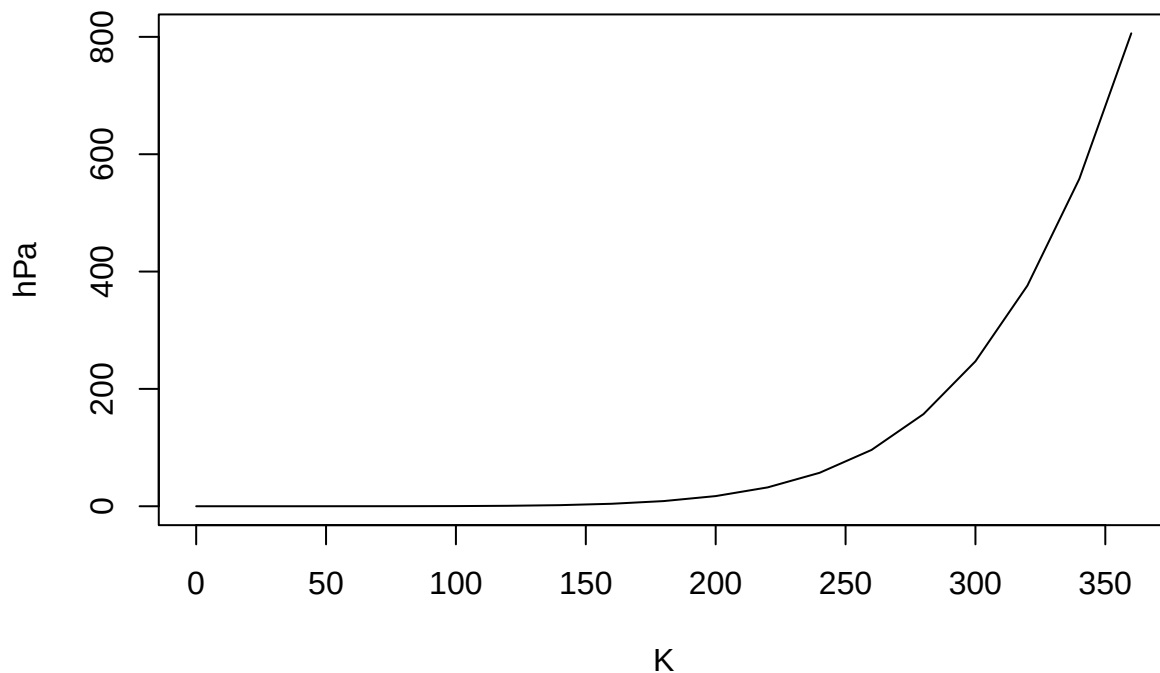
```
summary(cars)
```

```
##      speed      dist
##  Min.   : 4.0    Min.   : 2.00
##  1st Qu.:12.0    1st Qu.: 26.00
##  Median :15.0    Median : 36.00
##  Mean   :15.4    Mean   : 42.98
##  3rd Qu.:19.0    3rd Qu.: 56.00
##  Max.   :25.0    Max.   :120.00
```

Comando Plot

Nos permite graficar datos deseados, pudiendo asignar el nombre de las variables, hacer cambios al grafico. Para saber utilizar el comando escribimos `?plot` en la consola y podremos abrir una ventana explicativa, ahi tambien podemos ver los tipos de lineas que hay y que letra usar para cada una.

Presion del gas ideal



Funciones estadísticas

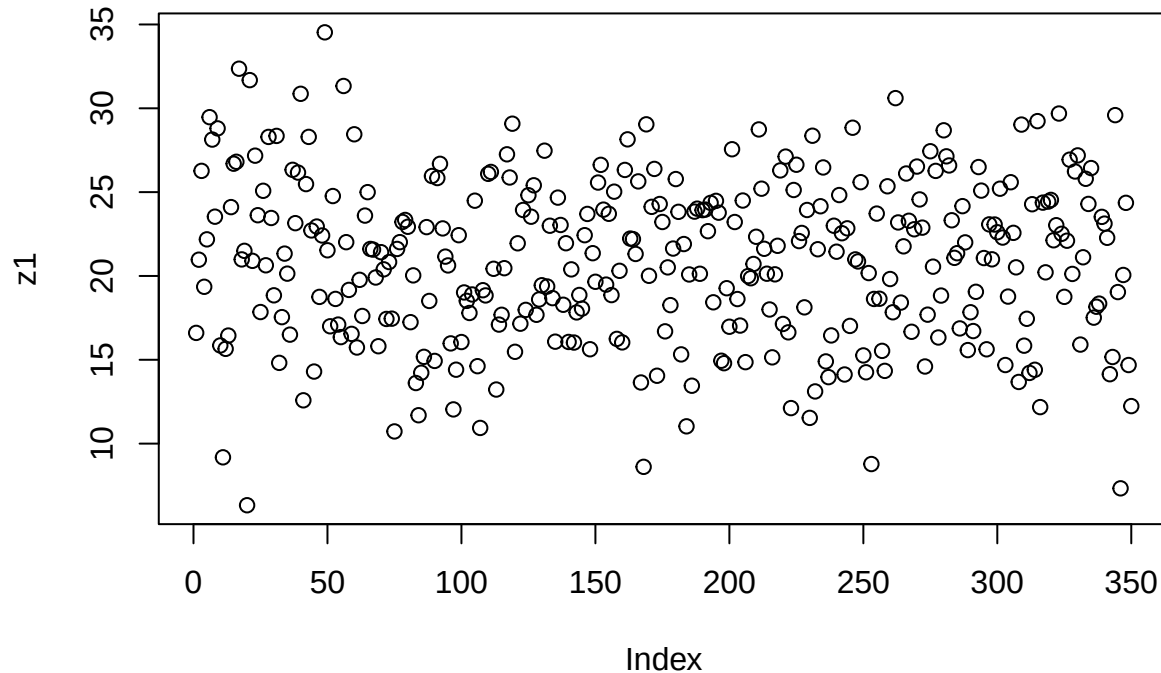
Vemos en todos estos graficos, 350 valores, valor promedio 21 y un desvio estandar de 5. El comando `rnorm`, nos da la distribución normal para el lenguaje R. En caso de tener dudas, escribir en la consola `?rnorm`

```
z1<-rnorm(350,21,5)
z1
```

```
## [1] 16.603787 20.970276 26.267954 19.353430 22.182736 29.471098 28.135211
## [8] 23.535353 28.801817 15.854179 9.181767 15.651791 16.444268 24.106733
## [15] 26.696258 26.810646 32.362450 20.987544 21.498551 6.324261 31.680202
## [22] 20.900931 27.175075 23.623471 17.847697 25.079219 20.636106 28.293289
## [29] 23.457667 18.844311 28.353834 14.816550 17.544274 21.333203 20.138063
## [36] 16.500852 26.330379 23.149241 26.166662 30.865077 12.583084 25.479486
## [43] 28.293906 22.715419 14.297482 22.947381 18.751538 22.414198 34.530760
## [50] 21.532144 16.997817 24.766519 18.630026 17.100521 16.352913 31.331426
## [57] 22.015283 19.167165 16.545486 28.451126 15.732588 19.769280 17.612563
## [64] 23.600124 24.998728 21.611258 21.572447 19.906723 15.809034 21.417790
## [71] 20.393724 17.435266 20.830573 17.453306 10.726503 21.611586 22.016876
## [78] 23.220422 23.339201 22.933875 17.242409 20.034776 13.612982 11.691228
## [85] 14.226755 15.175474 22.913771 18.508851 25.960955 14.932202 25.839640
## [92] 26.688321 22.835944 21.152277 20.628391 15.976437 12.040599 14.409388
## [99] 22.432430 16.057479 19.016922 18.513303 17.792608 18.895372 24.490076
## [106] 14.619154 10.934213 19.150807 18.838919 26.092086 26.204121 20.426786
## [113] 13.225405 17.100503 17.683494 20.464191 27.255258 25.873601 29.081968
## [120] 15.475631 21.948517 17.152669 23.937108 17.976315 24.816603 23.533289
## [127] 25.407756 17.683808 18.598261 19.445706 27.472252 19.371576 22.995952
## [134] 18.678567 16.081112 24.673256 23.041100 18.281872 21.952488 16.063406
## [141] 20.391909 16.027645 17.825030 18.865355 18.050625 22.432084 23.698841
## [148] 15.628733 21.358618 19.652605 25.571314 26.627121 23.955439 19.483970
```

```
## [155] 23.706885 18.859243 25.027180 16.245391 20.307015 16.036935 26.323293
## [162] 28.148807 22.228937 22.199397 21.312097 25.640454 13.645121 8.614258
## [169] 29.043015 20.009242 24.122220 26.384072 14.048016 24.291726 23.218459
## [176] 16.695784 20.511384 18.260106 21.634603 25.779316 23.824659 15.321226
## [183] 21.903033 11.031043 20.088248 13.452682 23.838528 24.014297 20.130781
## [190] 23.926624 23.959205 22.669862 24.359422 18.428408 24.479154 23.774174
## [197] 14.942671 14.796237 19.269356 16.971420 27.557624 23.216645 18.622195
## [204] 17.040586 24.500071 14.858572 19.993752 19.864810 20.713312 22.337403
## [211] 28.740681 25.205831 21.620529 20.135493 17.999991 15.145081 20.097004
## [218] 21.798752 26.294954 17.135870 27.118589 16.636553 12.119016 25.128329
## [225] 26.633672 22.079812 22.565280 18.127383 23.935020 11.534833 28.364639
## [232] 13.112941 21.593849 24.157790 26.471996 14.902104 13.964161 16.453142
## [239] 22.995731 21.447226 24.825589 22.549504 14.122870 22.840557 17.020176
## [246] 28.841595 20.987008 20.864907 25.594373 15.260353 14.254687 20.172336
## [253] 8.781788 18.627773 23.719747 18.640571 15.531459 14.338996 25.348088
## [260] 19.817448 17.833372 30.607428 23.192419 18.412422 21.767120 26.110458
## [267] 23.292914 16.670999 22.784881 26.526351 24.565125 22.881484 14.599967
## [274] 17.691834 27.439146 20.554900 26.267053 16.330450 18.830422 28.687206
## [281] 27.134498 26.590521 23.321031 21.079209 21.357718 16.870584 24.163134
## [288] 22.002384 15.577168 17.834561 16.714114 19.054408 26.495271 25.090981
## [295] 21.068096 15.625574 23.076583 20.992752 23.067410 22.606189 25.204056
## [302] 22.290799 14.687632 18.770159 25.589817 22.571454 20.514088 13.677374
## [309] 29.029247 15.841416 17.445400 14.216949 24.277645 14.403860 29.235118
## [316] 12.176472 24.373672 20.215392 24.436163 24.535998 22.127622 23.026116
## [323] 29.688315 22.514308 18.754183 22.102580 26.936087 20.120674 26.231319
## [330] 27.191426 15.906963 21.114586 25.795016 24.295272 26.435714 17.513244
## [337] 18.164809 18.349971 23.523970 23.122402 22.275801 14.140168 15.165777
## [344] 29.593246 19.038158 7.332475 20.059247 24.361301 14.687453 12.237345
```

```
plot(z1)
```



```
w1<-length(z1)
w1
```

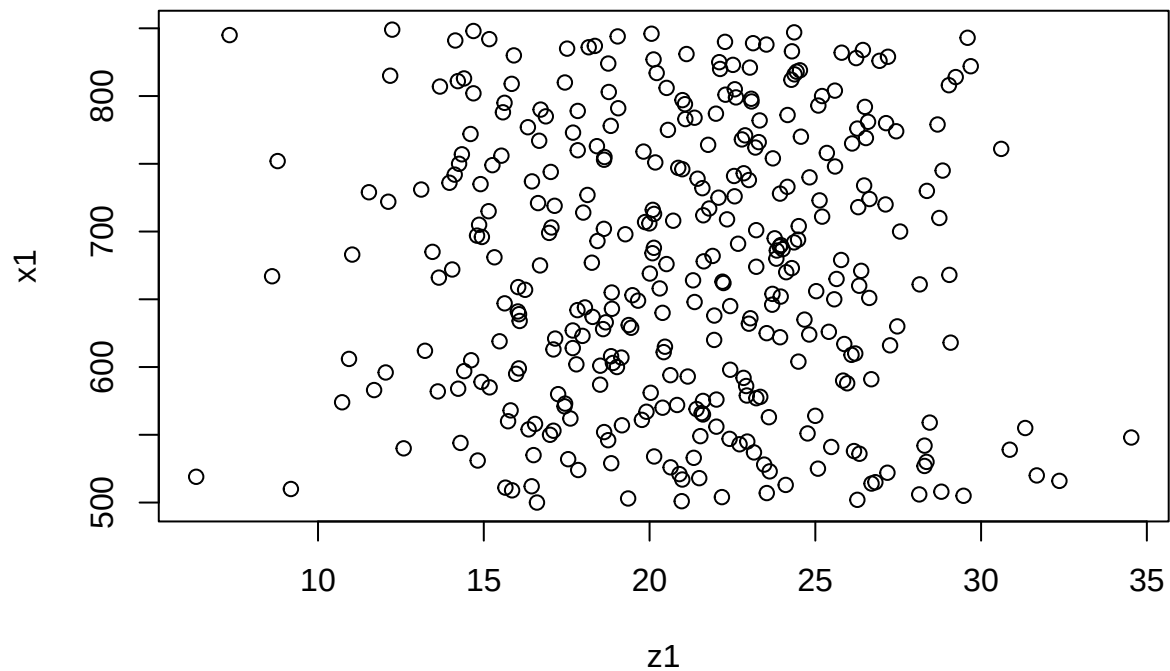
```
## [1] 350
```

```
x1<-(500:849)
```

```
x1
```

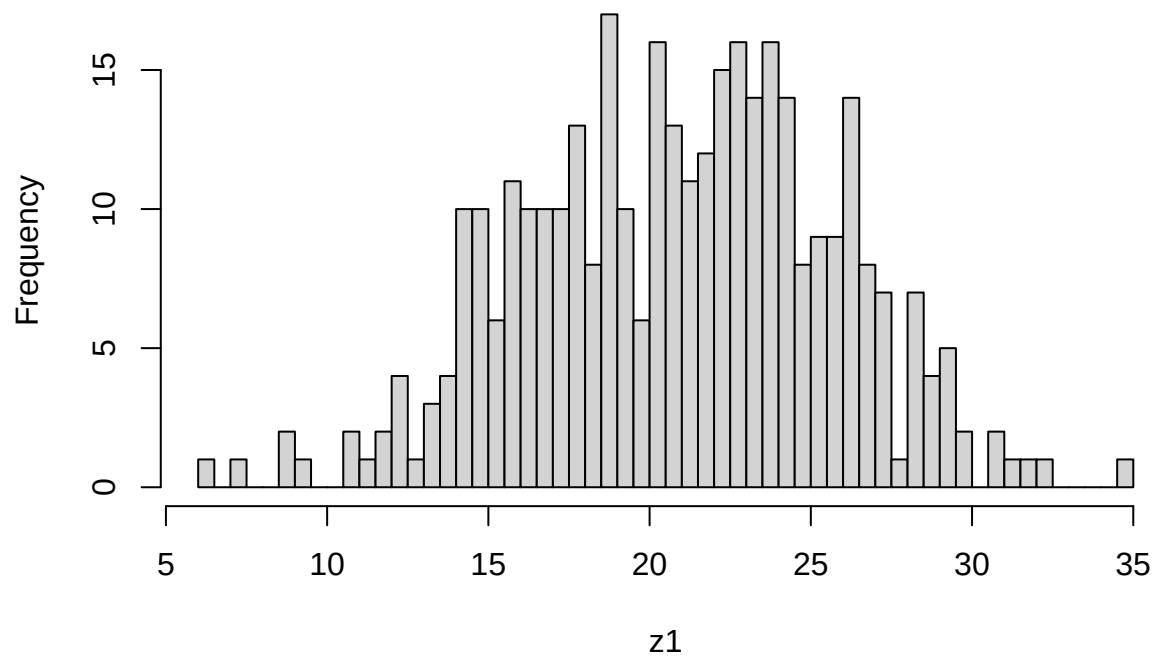
```
## [1] 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517
## [19] 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535
## [37] 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553
## [55] 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571
## [73] 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589
## [91] 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607
## [109] 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625
## [127] 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643
## [145] 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661
## [163] 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679
## [181] 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697
## [199] 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715
## [217] 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733
## [235] 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751
## [253] 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769
## [271] 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787
## [289] 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805
## [307] 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823
## [325] 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841
## [343] 842 843 844 845 846 847 848 849
```

```
plot(z1,x1)
```



```
hist(z1,main="Histograma de edades",breaks=50)
```

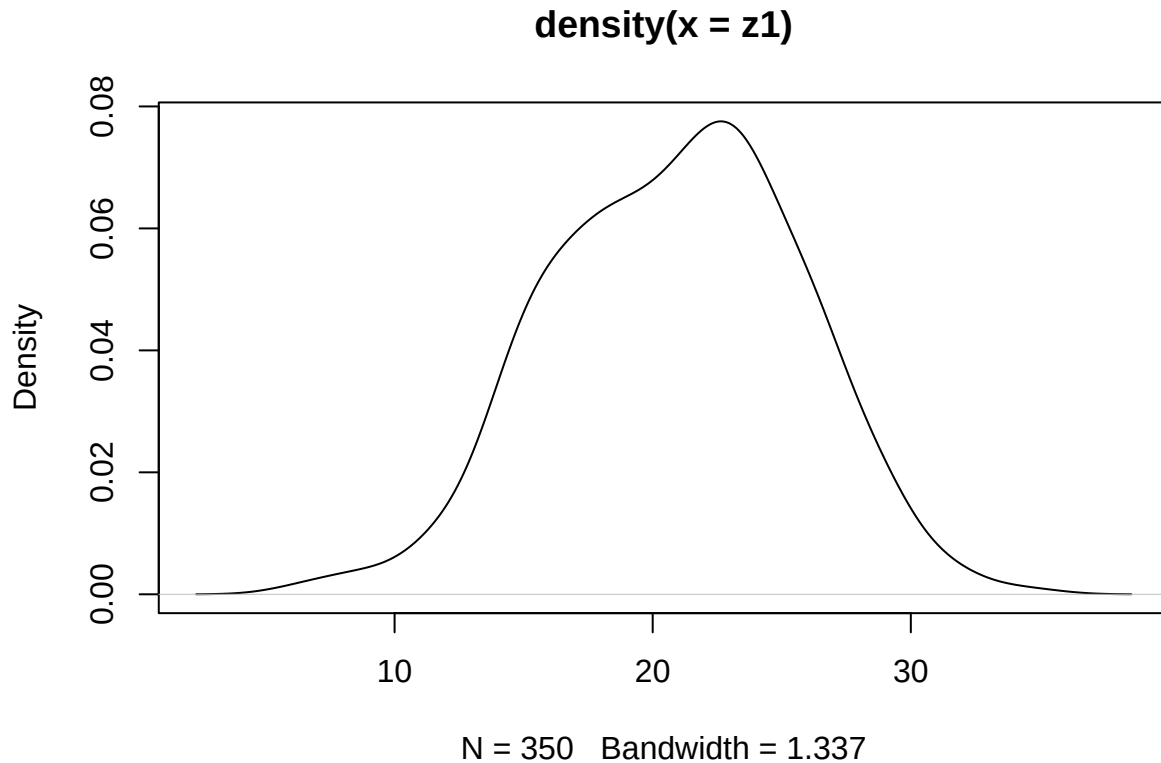
Histograma de edades



```
density(z1)
```

```
##  
## Call:  
## density.default(x = z1)  
##  
## Data: z1 (350 obs.); Bandwidth 'bw' = 1.337  
##  
##      x          y  
## Min.   : 2.314   Min.   :9.542e-06  
## 1st Qu.:11.371   1st Qu.:1.976e-03  
## Median :20.428   Median :1.385e-02  
## Mean   :20.428   Mean   :2.755e-02  
## 3rd Qu.:29.484   3rd Qu.:5.708e-02  
## Max.   :38.541   Max.   :7.756e-02
```

```
plot(density(z1))
```



Note that the `echo = FALSE` parameter was added to the code chunk to prevent printing of the R code that generated the plot.

Ejercicio 1 de algoritmos

Consigna: Las últimas 3 cifras de mi DNI son 546, crear una variable que tenga ese numero

```
DNI=546
```

Consigna: Crear un vector que tenga todos los numeros enteros desde el 1 al 546.

```
Secuencia_dni<-(1:546)
Secuencia_dni
```

```
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18
## [19] 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36
## [37] 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54
## [55] 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72
## [73] 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90
## [91] 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108
## [109] 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126
## [127] 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144
## [145] 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162
## [163] 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180
## [181] 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198
## [199] 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216
## [217] 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234
## [235] 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252
## [253] 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270
## [271] 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288
## [289] 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306
## [307] 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324
```

```
## [325] 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342
## [343] 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360
## [361] 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378
## [379] 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396
## [397] 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414
## [415] 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432
## [433] 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450
## [451] 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468
## [469] 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486
## [487] 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504
## [505] 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522
## [523] 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540
## [541] 541 542 543 544 545 546
```

Ejercicio 2:

Calcular la suma de todos los valores del vector Secuencia_dni

```
total<-0
Valor_final<-length(Secuencia_dni)
for(i in 1:Valor_final) {
total<-total+i
}
total
```

```
## [1] 149331
```

Ejercicio 3

Repetir el ejercicio anterior en Phyton

```
suma = 0
for i in range(1,546):
    suma += i
    print(suma)
```

```
## 1
## 3
## 6
## 10
## 15
## 21
## 28
## 36
## 45
## 55
## 66
## 78
## 91
## 105
## 120
## 136
## 153
## 171
## 190
## 210
```

231
253
276
300
325
351
378
406
435
465
496
528
561
595
630
666
703
741
780
820
861
903
946
990
1035
1081
1128
1176
1225
1275
1326
1378
1431
1485
1540
1596
1653
1711
1770
1830
1891
1953
2016
2080
2145
2211
2278
2346
2415
2485
2556
2628
2701
2775

2850
2926
3003
3081
3160
3240
3321
3403
3486
3570
3655
3741
3828
3916
4005
4095
4186
4278
4371
4465
4560
4656
4753
4851
4950
5050
5151
5253
5356
5460
5565
5671
5778
5886
5995
6105
6216
6328
6441
6555
6670
6786
6903
7021
7140
7260
7381
7503
7626
7750
7875
8001
8128
8256

8385
8515
8646
8778
8911
9045
9180
9316
9453
9591
9730
9870
10011
10153
10296
10440
10585
10731
10878
11026
11175
11325
11476
11628
11781
11935
12090
12246
12403
12561
12720
12880
13041
13203
13366
13530
13695
13861
14028
14196
14365
14535
14706
14878
15051
15225
15400
15576
15753
15931
16110
16290
16471
16653

16836
17020
17205
17391
17578
17766
17955
18145
18336
18528
18721
18915
19110
19306
19503
19701
19900
20100
20301
20503
20706
20910
21115
21321
21528
21736
21945
22155
22366
22578
22791
23005
23220
23436
23653
23871
24090
24310
24531
24753
24976
25200
25425
25651
25878
26106
26335
26565
26796
27028
27261
27495
27730
27966

28203
28441
28680
28920
29161
29403
29646
29890
30135
30381
30628
30876
31125
31375
31626
31878
32131
32385
32640
32896
33153
33411
33670
33930
34191
34453
34716
34980
35245
35511
35778
36046
36315
36585
36856
37128
37401
37675
37950
38226
38503
38781
39060
39340
39621
39903
40186
40470
40755
41041
41328
41616
41905
42195

42486
42778
43071
43365
43660
43956
44253
44551
44850
45150
45451
45753
46056
46360
46665
46971
47278
47586
47895
48205
48516
48828
49141
49455
49770
50086
50403
50721
51040
51360
51681
52003
52326
52650
52975
53301
53628
53956
54285
54615
54946
55278
55611
55945
56280
56616
56953
57291
57630
57970
58311
58653
58996
59340

59685
60031
60378
60726
61075
61425
61776
62128
62481
62835
63190
63546
63903
64261
64620
64980
65341
65703
66066
66430
66795
67161
67528
67896
68265
68635
69006
69378
69751
70125
70500
70876
71253
71631
72010
72390
72771
73153
73536
73920
74305
74691
75078
75466
75855
76245
76636
77028
77421
77815
78210
78606
79003
79401

79800
80200
80601
81003
81406
81810
82215
82621
83028
83436
83845
84255
84666
85078
85491
85905
86320
86736
87153
87571
87990
88410
88831
89253
89676
90100
90525
90951
91378
91806
92235
92665
93096
93528
93961
94395
94830
95266
95703
96141
96580
97020
97461
97903
98346
98790
99235
99681
100128
100576
101025
101475
101926
102378

102831
103285
103740
104196
104653
105111
105570
106030
106491
106953
107416
107880
108345
108811
109278
109746
110215
110685
111156
111628
112101
112575
113050
113526
114003
114481
114960
115440
115921
116403
116886
117370
117855
118341
118828
119316
119805
120295
120786
121278
121771
122265
122760
123256
123753
124251
124750
125250
125751
126253
126756
127260
127765
128271


```
## 128778
## 129286
## 129795
## 130305
## 130816
## 131328
## 131841
## 132355
## 132870
## 133386
## 133903
## 134421
## 134940
## 135460
## 135981
## 136503
## 137026
## 137550
## 138075
## 138601
## 139128
## 139656
## 140185
## 140715
## 141246
## 141778
## 142311
## 142845
## 143380
## 143916
## 144453
## 144991
## 145530
## 146070
## 146611
## 147153
## 147696
## 148240
## 148785
```

Ejercicio 4

Cuánto tarda en correr el código del ejercicio 2?

```
inicio<-Sys.time()
total<-0
Valor_final<-length(Secuencia_dni)
for(i in 1:Valor_final){
  total<-total+i
}
total
```

```
## [1] 149331
```

```
final<-Sys.time()
final-inicio
```

```
## Time difference of 0.003855944 secs
```

Ejercicio 5

Resolver el ejercicio anterior utilizando la cabeza de Gauss

Ejercicios profe:

1.4 Generar un vector secuencia

```
# Generar secuencia con bucle 'for'
# Primero inicializamos el vector
system.time({
  A <- 0
  for (i in 1:50000) {
    A[i] <- (i * 2)
  }
})
```

```
##      user      system elapsed
##    0.012    0.000    0.012
```

```
# Verificamos los resultados
head(A)
```

```
## [1]  2  4  6  8 10 12
```

```
tail(A)
```

```
## [1] 99990 99992 99994 99996 99998 100000
```

```
system.time({
  B <- seq(from = 2, to = 100000, by = 2)
})
```

```
##      user      system elapsed
##    0.001    0.000    0.000
```

```
# Verificamos los resultados
head(B)
```

```
## [1]  2  4  6  8 10 12
```

```
tail(B)
```

```
## [1] 99990 99992 99994 99996 99998 100000
```

1.5 Implementación de una serie Fibonacci o Fibonacci

```
# Definimos la cantidad de elementos de la serie que queremos calcular
n <- 30
```

```
# Método 1: Bucle 'for' con pre-asignación
system.time({
  fib <- numeric(n)
```

```

fib[1] <- 0
fib[2] <- 1

for (i in 3:n) {
  fib[i] <- fib[i-1] + fib[i-2]
}
})

##      user      system elapsed
##    0.003      0.000      0.003

# Ver los primeros resultados
print("Primeros números de la serie:")

## [1] "Primeros números de la serie:"
head(fib, 10)

## [1] 0 1 1 2 3 5 8 13 21 34

# Método 2: Función Recursiva

fib_recursiva <- function(k) {
  if (k == 0) return(0)
  if (k == 1) return(1)
  return(fib_recursiva(k - 1) + fib_recursiva(k - 2))
}

print("Tiempo ejecución recursiva:")

## [1] "Tiempo ejecución recursiva:"

system.time({
  # Calculamos el elemento n de forma individual
  resultado_rec <- fib_recursiva(n)
})

##      user      system elapsed
##    1.241      0.001      1.249

```

1.6 Definición matemática recurrente

```

# --- Resolver consigna con bucle 'while' ---

# Inicializamos la serie según la definición matemática
fib <- c(0, 1)
i <- 2 # Empezamos el conteo de iteraciones/posiciones

system.time({
  # El bucle sigue mientras el último número generado sea <= 1.000.000
  while (fib[i] <= 1000000) {
    i <- i + 1
    # Definición recurrente:  $f(n+1) = f(n) + f(n-1)$ 
    fib[i] <- fib[i-1] + fib[i-2]
  }
})

```

```
##      user  system elapsed
##    0.003   0.000   0.004

# --- Resultados ---
cat("El primer número mayor a 1.000.000 es:", fib[i], "\n")

## El primer número mayor a 1.000.000 es: 1346269
cat("Se necesitaron", i - 1, "iteraciones (pasos) para superarlo.\n")

## Se necesitaron 31 iteraciones (pasos) para superarlo.
# Mostramos los últimos elementos para verificar
tail(fib)

## [1] 121393 196418 317811 514229 832040 1346269
# 1.7 Ordenacion de un vector por metodo burbuja
set.seed(123)
vector_desordenado <- sample(1:1000, 1000)

# --- 2. Implementación del Método Burbuja ---
bubble_sort <- function(x) {
  n <- length(x)
  # El bucle externo recorre todo el vector
  for (i in 1:(n - 1)) {
    # El bucle interno compara pares adyacentes
    for (j in 1:(n - i)) {
      if (x[j] > x[j + 1]) {
        # Intercambio de posición (el "burbujeo")
        temp <- x[j]
        x[j] <- x[j + 1]
        x[j + 1] <- temp
      }
    }
  }
  return(x)
}

# --- 3. Medimos Performance del Burbuja vs R Nativo ---
cat("Tiempo con Método Burbuja:\n")

## Tiempo con Método Burbuja:
system.time({
  vector_ordenado_burbuja <- bubble_sort(vector_desordenado)
})

##      user  system elapsed
##    0.059   0.000   0.060
cat("\nTiempo con función sort() nativa de R:\n")

##
## Tiempo con función sort() nativa de R:
```

```
system.time({
  vector_ordenado_R <- sort(vector_desordenado)
})
```

```
##      user  system elapsed
##         0         0         0
```

1.8 La penitencia de Newton

```
# Límite de la suma ( $1 \cdot 10^7$ )
n <- 1e7
```

```
# --- MÉTODO 1: (Bucle For) ---
```

```
system.time({
  suma_iterativa <- 0
  for (i in 1:n) {
    suma_iterativa <- suma_iterativa + i
  }
})
```

```
##      user  system elapsed
##    0.155    0.000    0.155
```

```
cat("Resultado Suma Iterativa:", suma_iterativa, "\n")
```

```
## Resultado Suma Iterativa: 5e+13
```

```
# --- MÉTODO 2: La "Estrategia de Newton/Gauss" (Fórmula Analítica) ---
```

```
 #(n * (n + 1)) / 2
```

```
system.time({
  suma_analitica <- (n * (n + 1)) / 2
})
```

```
##      user  system elapsed
##         0         0         0
```

```
cat("Resultado Suma Analítica:", suma_analitica, "\n")
```

```
## Resultado Suma Analítica: 5e+13
```

2 Gráfico de violín

```
library(ggplot2)
library(cluster)
```

```
set.seed(123)
```

```
vibracion <- c(rnorm(100, 2, 0.5), rnorm(100, 5, 0.8), rnorm(100, 8, 0.6))
```

```
temperatura <- c(rnorm(100, 30, 5), rnorm(100, 50, 7), rnorm(100, 70, 5))
```

```
df <- data.frame(vibracion, temperatura)
```

```
# Escalamiento
```

```
df_scaled <- scale(df)
```

```
# 2. Ejecución de K-means (k=3)
```

```

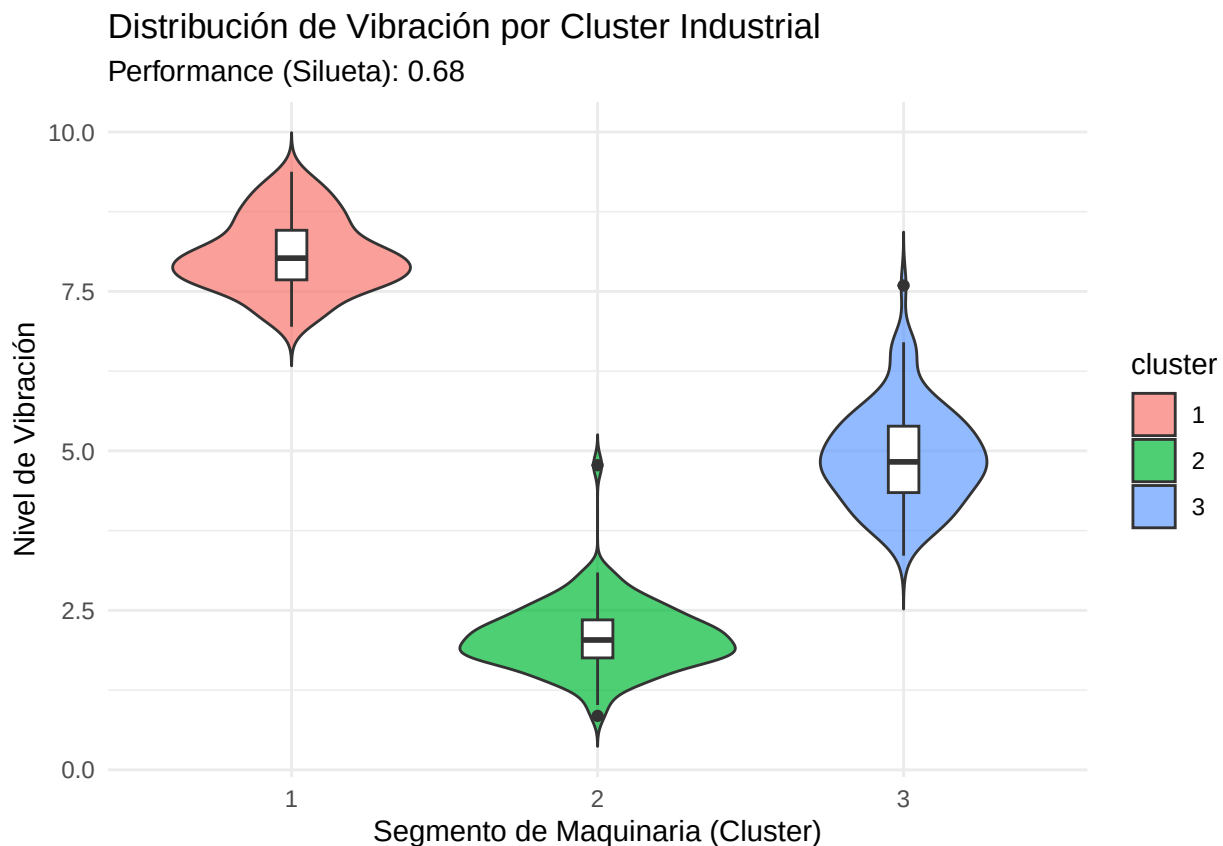
fit <- kmeans(df_scaled, centers = 3, nstart = 25)
df$cluster <- as.factor(fit$cluster)

# 3. Medición de Performance: Coeficiente de Silueta
sil <- silhouette(fit$cluster, dist(df_scaled))
avg_sil <- mean(sil[, 3])
cat("El coeficiente de silueta promedio es:", avg_sil)

## El coeficiente de silueta promedio es: 0.6808356

# 4. Visualización con Gráfico de Violín
ggplot(df, aes(x = cluster, y = vibracion, fill = cluster)) +
  geom_violin(trim = FALSE, alpha = 0.7) +
  geom_boxplot(width = 0.1, fill = "white") +
  labs(title = "Distribución de Vibración por Cluster Industrial",
        subtitle = paste("Performance (Silueta):", round(avg_sil, 2)),
        x = "Segmento de Maquinaria (Cluster)",
        y = "Nivel de Vibración") +
  theme_minimal()

```



Ejercicio Matrices e Inversa

En este ejercicio, crearemos una matriz cuadrada de 3x3 utilizando el valor de tu DNI (546). Luego, usamos la función `solve()`

```

val <- 546

datos <- c(val, 10, 15,
           2, 5, 8,

```

```

7, 1, 9)

A <- matrix(datos, nrow = 3, ncol = 3, byrow = TRUE)

cat("### Matriz Original (A):\n")

## ### Matriz Original (A):
print(A)

##      [,1] [,2] [,3]
## [1,] 546   10   15
## [2,]  2    5    8
## [3,]  7    1    9

determinante <- det(A)
cat("\nDeterminante de la matriz:", determinante, "\n")

##
## Determinante de la matriz: 20087

if (determinante != 0) {
  A_inversa <- solve(A)
  cat("\n### Matriz Inversa (A^-1):\n")
  print(A_inversa)

  identidad <- round(A %*% A_inversa, 2)
  cat("\n### Verificación (A * A^-1 = Identidad):\n")
  print(identidad)
} else {
  cat("\nLa matriz es singular y no tiene inversa.")
}

##
## ### Matriz Inversa (A^-1):
##      [,1]      [,2]      [,3]
## [1,] 0.001841987 -0.003733758 0.0002489172
## [2,] 0.001891771 0.239408573 -0.2159605715
## [3,] -0.001642854 -0.023696918 0.1349131279
##
## ### Verificación (A * A^-1 = Identidad):
##      [,1] [,2] [,3]
## [1,]  1   0   0
## [2,]  0   1   0
## [3,]  0   0   1

```